# CS-462 – Programming Language Foundations
# Lecture #14: Polymorphism (System F)

EPFL

## 1 Parametric Polymorphism

In the simply-typed $\lambda$-calculus, every function argument must be annotated with a type. As we saw when we discussed type completeness, this means that there is a different identity function for each type: $\lambda x : \mathbf{int}.\, x$, $\lambda x : \mathbf{bool}.\, x$, $\lambda x : \mathbf{int} \to \mathbf{int}.\, x$, and so on. Each of these functions does exactly the same thing, but we have to write a separate copy for each type.

*Polymorphism* (Greek for "many forms") is the ability for code to be used with values of different types. For example, a polymorphic function is one that can be invoked with arguments of different types, and a polymorphic data type is one that can contain elements of different types. There are several kinds of polymorphism that are commonly used in modern programming languages.

- *Subtype polymorphism* allows a term to have many types, using a subsumption rule that lets a value of type $\tau$ masquerade as a value of type $\tau'$, provided that $\tau$ is a subtype of $\tau'$.
- *Ad-hoc polymorphism* usually refers to code that appears to be polymorphic to the programmer, but whose implementation is not. For example, languages with *overloading* allow the same function name to be used for functions that take different types of parameters. Although it looks like a polymorphic function to the code that uses it, there are actually several implementations (none of them polymorphic), and the compiler invokes the appropriate one. Ad-hoc polymorphism is a dispatch mechanism: the types of the arguments determine (either at compile time or at run time) which code to invoke.
- *Parametric polymorphism* refers to code that is written without knowledge of the actual types of its arguments: the code is *parametric* in those types. Examples include polymorphic functions in OCaml and generics in Java, Scala, and Rust.

In this lecture, we will study parametric polymorphism in detail.

As a motivating example, recall the function that applies a function $f$ twice to an argument $x$. In the simply-typed $\lambda$-calculus, we can write it for integers:

$$\mathsf{twiceInt} \triangleq \lambda f : \mathbf{int} \to \mathbf{int}.\, \lambda x : \mathbf{int}.\, f\ (f\ x)$$

But we must write it again for Booleans, and again for functions on integers, and so on:

$$\mathsf{twiceBool} \triangleq \lambda f : \mathbf{bool} \to \mathbf{bool}.\, \lambda x : \mathbf{bool}.\, f\ (f\ x)$$
$$\mathsf{twiceFn} \triangleq \lambda f : (\mathbf{int} \to \mathbf{int}) \to (\mathbf{int} \to \mathbf{int}).\, \lambda x : \mathbf{int} \to \mathbf{int}.\, f\ (f\ x)$$

This violates a fundamental principle of software engineering:

> **Abstraction Principle:** *Every major piece of functionality in a program should be implemented in just one place in the code. When similar functionality is provided by distinct pieces of code, the two should be combined into one by abstracting out the varying parts.*

In the twice functions, the varying parts are the types. We need a way to abstract over the type, and later instantiate it with different concrete types.

## 2 System F

We can extend the simply-typed $\lambda$-calculus with abstraction over types. The resulting language was invented independently by the logician Jean-Yves Girard in 1972, who called it *System F*, and by the computer scientist John Reynolds in 1974, who called it the *polymorphic $\lambda$-calculus*.

A *type abstraction*, written $\Lambda X.\, e$, is a function from types to terms: $\Lambda$ is the upper-case form of the Greek letter $\lambda$, and $X$ is a *type variable*. A *type application*, written $e\ [\tau]$, *instantiates* a type abstraction with a particular type $\tau$. For example, the polymorphic identity function is

$$\mathsf{ID} \triangleq \Lambda X.\, \lambda x : X.\, x.$$

When a type abstraction meets a type application during evaluation, we substitute the type for the type variable, just as $\beta$-reduction substitutes an argument for a variable.

### 2.1 Syntax and Operational Semantics

We start from the simply-typed $\lambda$-calculus with **unit**, integers, Booleans, and conditionals, and add type abstraction, type application, type variables, and polymorphic types $\forall X.\, \tau$:

$$\begin{aligned}
e &::= x \mid \lambda x : \tau.\, e \mid e_1\ e_2 \mid () \mid n \mid e_1 \oplus e_2 \mid \mathsf{true} \mid \mathsf{false} \mid \mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3 \mid \Lambda X.\, e \mid e\ [\tau] \\
v &::= \lambda x : \tau.\, e \mid () \mid n \mid \mathsf{true} \mid \mathsf{false} \mid \Lambda X.\, e \\
\tau &::= \mathbf{unit} \mid \mathbf{int} \mid \mathbf{bool} \mid \tau_1 \to \tau_2 \mid X \mid \forall X.\, \tau
\end{aligned}$$

Here $\oplus$ ranges over the arithmetic operators $+$, $-$, and $\times$, as before. The constructs $\Lambda X.\, e$ and $\forall X.\, \tau$ *bind* the type variable $X$ in $e$ and in $\tau$, just as $\lambda x : \tau.\, e$ binds the term variable $x$ in $e$. So we have the usual notions of free and bound type variables, and of $\alpha$-equivalence: for example, $\forall X.\, X \to X$ and $\forall Y.\, Y \to Y$ are the same type.

The evaluation rules are the same as for the simply-typed $\lambda$-calculus (with CBV evaluation, as before), together with a new evaluation context and a new rule for type application:

$$E ::= \cdots \mid E\ [\tau] \qquad\qquad \textsc{Type-}\beta\ \frac{}{(\Lambda X.\, e)\ [\tau] \to e\{\tau/X\}}$$

Note that $\Lambda X.\, e$ is a value: just as we do not evaluate under a $\lambda$, we do not evaluate under a $\Lambda$. For example, we can apply the polymorphic identity function to the type **int** and then to an integer:

$$\mathsf{ID}\ [\mathbf{int}]\ 5 = (\Lambda X.\, \lambda x : X.\, x)\ [\mathbf{int}]\ 5 \to (\lambda x : \mathbf{int}.\, x)\ 5 \to 5.$$

We can apply ID to other types as well. For example, ID [**int** $\to$ **int**] steps to the identity function on functions from integers to integers, $\lambda x : \mathbf{int} \to \mathbf{int}.\, x$.

Type substitution $\tau'\{\tau/X\}$ must avoid capturing free type variables, just like substitution for terms. For example, the type $\forall X.\, \forall Y.\, X \to Y \to X$ has a bound type variable $Y$. If we instantiate $X$ with a type that mentions a *different* type variable that happens to also be named $Y$, as in

$$\Lambda Y.\ (\Lambda X.\, \Lambda Y.\, \lambda x : X.\, \lambda y : Y.\, x)\ [Y],$$

then we must rename the inner bound variable first:

$$(\forall Y.\, X \to Y \to X)\{Y/X\} = \forall Y'.\, Y \to Y' \to Y.$$

The naive result $\forall Y.\, Y \to Y \to Y$ would be wrong: the outer $Y$ would be captured by the inner $\forall Y$.

## 2.2 Type System

The type of a type abstraction $\Lambda X.\, e$ is $\forall X.\, \tau$, where $\tau$ is the type of $e$, which may mention the type variable $X$. Intuitively, we use this notation because $e$ has type $\tau$ for *every* choice of the type $X$.

Type checking is slightly different than before. Besides the typing context $\Gamma$, which maps term variables to types, we also need to keep track of the set $\Delta$ of type variables that are in scope. This ensures that a type variable $X$ is only used within an enclosing type abstraction $\Lambda X.\, e$. For example, the expression $\lambda x : X.\, x$ should not be well-typed on its own, because $X$ is not bound. So typing judgments now have the form $\Delta, \Gamma \vdash e : \tau$. We also use an auxiliary judgment $\Delta \vdash \tau\ \mathsf{ok}$, which says that every free type variable of $\tau$ is in $\Delta$.

The typing rules for the constructs of the simply-typed $\lambda$-calculus are the same as before, except that they carry $\Delta$ along unchanged, and the type annotation on an abstraction must be well-formed. For example:

$$\textsc{T-Var}\ \frac{\Gamma(x) = \tau}{\Delta, \Gamma \vdash x : \tau} \qquad \textsc{T-Abs}\ \frac{\Delta \vdash \tau\ \mathsf{ok} \quad \Delta, \Gamma, x : \tau \vdash e : \tau'}{\Delta, \Gamma \vdash \lambda x : \tau.\, e : \tau \to \tau'}$$

$$\textsc{T-App}\ \frac{\Delta, \Gamma \vdash e_1 : \tau \to \tau' \quad \Delta, \Gamma \vdash e_2 : \tau}{\Delta, \Gamma \vdash e_1\ e_2 : \tau'} \qquad \textsc{T-If}\ \frac{\Delta, \Gamma \vdash e_1 : \mathbf{bool} \quad \Delta, \Gamma \vdash e_2 : \tau \quad \Delta, \Gamma \vdash e_3 : \tau}{\Delta, \Gamma \vdash \mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3 : \tau}$$

The rules for (), integers, arithmetic, and Booleans are similar. There are two new typing rules, one for type abstraction and one for type application:

$$\textsc{T-TAbs}\ \frac{\Delta \cup \{X\}, \Gamma \vdash e : \tau}{\Delta, \Gamma \vdash \Lambda X.\, e : \forall X.\, \tau} \qquad \textsc{T-TApp}\ \frac{\Delta, \Gamma \vdash e : \forall X.\, \tau' \quad \Delta \vdash \tau\ \mathsf{ok}}{\Delta, \Gamma \vdash e\ [\tau] : \tau'\{\tau/X\}}$$

In T-TAbs, we always choose the bound type variable $X$ so that $X \notin \Delta$, renaming it if necessary using $\alpha$-equivalence. Finally, here are the rules for type well-formedness:

$$\frac{X \in \Delta}{\Delta \vdash X\ \mathsf{ok}} \qquad \frac{}{\Delta \vdash \mathbf{unit}\ \mathsf{ok}} \qquad \frac{}{\Delta \vdash \mathbf{int}\ \mathsf{ok}} \qquad \frac{}{\Delta \vdash \mathbf{bool}\ \mathsf{ok}}$$

$$\frac{\Delta \vdash \tau_1\ \mathsf{ok} \quad \Delta \vdash \tau_2\ \mathsf{ok}}{\Delta \vdash \tau_1 \to \tau_2\ \mathsf{ok}} \qquad \frac{\Delta \cup \{X\} \vdash \tau\ \mathsf{ok}}{\Delta \vdash \forall X.\, \tau\ \mathsf{ok}}$$

For example, here is a derivation showing that the polymorphic identity function has type $\forall X.\, X \to X$:

$$\textsc{T-TAbs}\ \frac{\textsc{T-Abs}\ \dfrac{\dfrac{X \in \{X\}}{\{X\} \vdash X\ \mathsf{ok}} \quad \textsc{T-Var}\ \dfrac{(x : X)(x) = X}{\{X\}, x : X \vdash x : X}}{\{X\}, \cdot \vdash \lambda x : X.\, x : X \to X}}{\emptyset, \cdot \vdash \Lambda X.\, \lambda x : X.\, x : \forall X.\, X \to X}$$

Then, by T-TApp, ID [**int**] has type $(X \to X)\{\mathbf{int}/X\} = \mathbf{int} \to \mathbf{int}$.

# 3 Programming in System F

## 3.1 Twice

Returning to our motivating example, we can write a single polymorphic version of twice:

$$\mathsf{twice} \triangleq \Lambda X.\, \lambda f : X \to X.\, \lambda x : X.\, f\ (f\ x) \ :\ \forall X.\, (X \to X) \to X \to X$$

We can instantiate it at whatever type we need. For example:

$$\begin{aligned}\mathsf{twice}\ [\mathbf{int}]\ (\lambda n : \mathbf{int}.\, n + 1)\ 7 &\to (\lambda f : \mathbf{int} \to \mathbf{int}.\, \lambda x : \mathbf{int}.\, f\ (f\ x))\ (\lambda n : \mathbf{int}.\, n + 1)\ 7 \\ &\to^* 9\end{aligned}$$

and

$$\mathsf{twice}\ [\mathbf{bool}]\ (\lambda b : \mathbf{bool}.\, \mathsf{if}\ b\ \mathsf{then\ false\ else\ true})\ \mathsf{true} \to^* \mathsf{true}.$$

## 3.2 Self Application

In the simply-typed $\lambda$-calculus, we had no way of typing the expression $\lambda x.\, x\ x$. In System F, we can type it by giving $x$ a polymorphic type, and instantiating it at that same type:

$$\vdash \quad \lambda x : \forall X.\, X \to X.\, x\ [\forall X.\, X \to X]\ x \quad : \quad (\forall X.\, X \to X) \to (\forall X.\, X \to X)$$

For example, applying this function to ID returns ID. However, we still cannot type $\Omega$: as we will see below, every well-typed System F program terminates.

## 3.3 Church Numerals

In the untyped $\lambda$-calculus, the Church numeral $\overline{n}$ is a function that takes $f$ and $x$ and applies $f$ to $x$ $n$ times. In System F, all of the Church numerals have the same type:

$$\mathsf{NAT} \triangleq \forall X.\, (X \to X) \to X \to X$$

For example:

$$\begin{aligned}\overline{0} &\triangleq \Lambda X.\, \lambda f : X \to X.\, \lambda x : X.\, x \\ \overline{1} &\triangleq \Lambda X.\, \lambda f : X \to X.\, \lambda x : X.\, f\ x \\ \mathsf{SUCC} &\triangleq \lambda n : \mathsf{NAT}.\, \Lambda X.\, \lambda f : X \to X.\, \lambda x : X.\, f\ (n\ [X]\ f\ x)\end{aligned}$$

Note that SUCC has to instantiate $n$ at the type $X$ before applying it to $f$ and $x$. And note that twice is exactly $\overline{2}$!

## 3.4 Products and Sums

We can encode products and sums in System F without adding any new types, using the Church encodings from the untyped $\lambda$-calculus. A pair is a function that waits for a "consumer" $p$ and

hands it both components; the consumer gets to choose the result type $R$:

$$\begin{array}{rcl}
\tau_1 \times \tau_2 & \triangleq & \forall R.\, (\tau_1 \to \tau_2 \to R) \to R \\
\mathsf{pair} & \triangleq & \Lambda T_1.\, \Lambda T_2.\, \lambda v_1 : T_1.\, \lambda v_2 : T_2. \\
& & \qquad \Lambda R.\, \lambda p : T_1 \to T_2 \to R.\, p\ v_1\ v_2 \\
\mathsf{fst} & \triangleq & \Lambda T_1.\, \Lambda T_2.\, \lambda v : T_1 \times T_2.\, v\ [T_1]\ (\lambda x : T_1.\, \lambda y : T_2.\, x) \\
\mathsf{snd} & \triangleq & \Lambda T_1.\, \Lambda T_2.\, \lambda v : T_1 \times T_2.\, v\ [T_2]\ (\lambda x : T_1.\, \lambda y : T_2.\, y)
\end{array}$$

Similarly, a value of a sum type waits for two functions, one for each case, and calls the appropriate one:

$$\begin{array}{rcl}
\tau_1 + \tau_2 & \triangleq & \forall R.\, (\tau_1 \to R) \to (\tau_2 \to R) \to R \\
\mathsf{inl} & \triangleq & \Lambda T_1.\, \Lambda T_2.\, \lambda v : T_1.\, \Lambda R.\, \lambda b_1 : T_1 \to R.\, \lambda b_2 : T_2 \to R.\, b_1\ v \\
\mathsf{inr} & \triangleq & \Lambda T_1.\, \Lambda T_2.\, \lambda v : T_2.\, \Lambda R.\, \lambda b_1 : T_1 \to R.\, \lambda b_2 : T_2 \to R.\, b_2\ v \\
\mathsf{case} & \triangleq & \Lambda T_1.\, \Lambda T_2.\, \Lambda R.\, \lambda v : T_1 + T_2.\, \lambda b_1 : T_1 \to R.\, \lambda b_2 : T_2 \to R.\, v\ [R]\ b_1\ b_2
\end{array}$$

Even **unit** and **bool** are not really necessary: we could define **unit** $\triangleq \forall R.\, R \to R$, whose only value is ID, and **bool** $\triangleq$ **unit** + **unit**.

# 4 Existential Types

A universal type $\forall X.\, \tau$ says that *for every* type $X$, we can produce a value of type $\tau$: the code that uses the polymorphic value chooses $X$. Dually, an *existential type* $\exists X.\, \tau$ says that *there is some* type $X$ for which we have a value of type $\tau$: the code that creates the value chooses $X$, and the code that uses it does not know what $X$ is.

Existential types are exactly what we need to hide the implementation of an *abstract data type* behind an interface, as modules do in languages like OCaml. For example, here is an interface for counters:

$$\mathsf{COUNTER} \triangleq \exists C.\, C \times (C \to C) \times (C \to \mathbf{int})$$

That is, there is some type $C$ of counters, together with an initial counter, an operation to increment a counter, and an operation to read the current count. (In the examples in this section, we write $(e_1, e_2, e_3)$ for tuples and $\#i\ e$ for the projections. These can be added to the language directly, as in the simply-typed $\lambda$-calculus, or encoded as above.)

## 4.1 Syntax and Semantics

We extend the language with a construct for creating existential values and one for using them:

$$\begin{array}{l}
e ::= \cdots \mid \mathsf{pack}\ \{\tau_1, e\}\ \mathsf{as}\ \exists X.\, \tau_2 \mid \mathsf{unpack}\ \{X, x\} = e_1\ \mathsf{in}\ e_2 \\
v ::= \cdots \mid \mathsf{pack}\ \{\tau_1, v\}\ \mathsf{as}\ \exists X.\, \tau_2 \\
\tau ::= \cdots \mid \exists X.\, \tau
\end{array}$$

The construct $\mathsf{pack}\ \{\tau_1, e\}\ \mathsf{as}\ \exists X.\, \tau_2$, called *packing*, combines a *witness type* $\tau_1$ with a value of type $\tau_2\{\tau_1/X\}$. The construct $\mathsf{unpack}\ \{X, x\} = e_1\ \mathsf{in}\ e_2$, called *unpacking*, opens the package $e_1$, binding its hidden type to $X$ and its value to $x$ in $e_2$. As usual, $\exists X.\, \tau$ binds $X$ in $\tau$, and $\mathsf{unpack}\ \{X, x\} = e_1\ \mathsf{in}\ e_2$ binds $X$ and $x$ in $e_2$.

The operational semantics evaluates the package and then substitutes its components into the body of the unpack:

$$E ::= \cdots \mid \mathsf{pack}\ \{\tau_1, E\}\ \mathsf{as}\ \exists X.\ \tau_2 \mid \mathsf{unpack}\ \{X, x\} = E\ \mathsf{in}\ e$$

$$\frac{}{\mathsf{unpack}\ \{X, x\} = (\mathsf{pack}\ \{\tau_1, v\}\ \mathsf{as}\ \exists Y.\ \tau_2)\ \mathsf{in}\ e \longrightarrow e\{v/x\}\{\tau_1/X\}}$$

### 4.2 Typing

The typing rules are as follows:

$$\text{T-Pack}\ \frac{\Delta \vdash \tau_1\ \mathsf{ok} \quad \Delta, \Gamma \vdash e : \tau_2\{\tau_1/X\}}{\Delta, \Gamma \vdash \mathsf{pack}\ \{\tau_1, e\}\ \mathsf{as}\ \exists X.\ \tau_2 : \exists X.\ \tau_2}$$

$$\text{T-Unpack}\ \frac{\Delta, \Gamma \vdash e_1 : \exists X.\ \tau_1 \quad \Delta \cup \{X\}, \Gamma, x : \tau_1 \vdash e_2 : \tau_2 \quad \Delta \vdash \tau_2\ \mathsf{ok}}{\Delta, \Gamma \vdash \mathsf{unpack}\ \{X, x\} = e_1\ \mathsf{in}\ e_2 : \tau_2}$$

In T-Pack, the annotation $\exists X.\ \tau_2$ tells the type checker which occurrences of the witness type $\tau_1$ should be hidden. Without it, we would not know whether, say, a value of type $\mathbf{int} \to \mathbf{int}$ should be given type $C \to C$, $C \to \mathbf{int}$, $\mathbf{int} \to C$, or $\mathbf{int} \to \mathbf{int}$. In T-Unpack, the body $e_2$ is checked with $X$ as a new type variable, about which nothing is known, so it must work for *any* witness type. (As in T-TAbs, we choose $X \notin \Delta$.) The premise $\Delta \vdash \tau_2\ \mathsf{ok}$ ensures that $X$ does not appear in the type of the result, so the hidden type cannot escape from the unpack.

### 4.3 Example: Counters

Here is an implementation of counters, using **int** as the witness type:

$$\mathsf{counter} \triangleq \mathsf{pack}\ \{\mathbf{int}, (0,\ \lambda i{:}\mathbf{int}.\ i + 1,\ \lambda i{:}\mathbf{int}.\ i)\}\ \mathsf{as}\ \exists C.\ C \times (C \to C) \times (C \to \mathbf{int})$$

A client can create a counter, increment it twice, and read the result:

$$\mathsf{unpack}\ \{C, c\} = \mathsf{counter}\ \mathsf{in}\ \ (\#3\ c)\ ((\#2\ c)\ ((\#2\ c)\ (\#1\ c))) \ \to^*\ 2$$

Inside the unpack, the client only knows that $c : C \times (C \to C) \times (C \to \mathbf{int})$. It cannot use the fact that $C$ is really **int**: for example, $(\#1\ c) + 1$ is not well-typed. And the expression $\mathsf{unpack}\ \{C, c\} = \mathsf{counter}\ \mathsf{in}\ \#1\ c$ is rejected by T-Unpack, since its type would be $C$, which is not in scope outside of the unpack.

Because clients can only use a package through its interface, we can replace one implementation with another without affecting any client. For example, this implementation counts down internally:

$$\mathsf{counter}' \triangleq \mathsf{pack}\ \{\mathbf{int}, (0,\ \lambda i{:}\mathbf{int}.\ i - 1,\ \lambda i{:}\mathbf{int}.\ 0 - i)\}\ \mathsf{as}\ \exists C.\ C \times (C \to C) \times (C \to \mathbf{int})$$

No client can distinguish counter from counter′. This property, called *representation independence*, is another consequence of parametricity.

### 4.4 Encoding Existentials

Like products and sums, existentials can be encoded in System F using universal types. The idea is that a package is a function that takes a result type $Y$ and a client that works for *any* hidden type, and calls the client with its witness type and value:

$$\begin{aligned} \exists X.\, \tau &\triangleq \forall Y.\, (\forall X.\, \tau \to Y) \to Y \\ \mathsf{pack}\ \{\tau_1, e\}\ \mathsf{as}\ \exists X.\, \tau_2 &\triangleq \Lambda Y.\, \lambda f : \forall X.\, \tau_2 \to Y.\, f\ [\tau_1]\ e \\ \mathsf{unpack}\ \{X, x\} = e_1\ \mathsf{in}\ e_2 &\triangleq e_1\ [\tau_2]\ (\Lambda X.\, \lambda x : \tau_1.\, e_2) \end{aligned}$$

where $e_1 : \exists X.\, \tau_1$ and $e_2 : \tau_2$. Note how the requirement that $X$ not appear free in $\tau_2$ shows up in the encoding: the client $\Lambda X.\, \lambda x : \tau_1.\, e_2$ must have type $\forall X.\, \tau_1 \to \tau_2$, and the result of instantiating $e_1$ at $\tau_2$ only makes sense if $\tau_2$ does not mention $X$.

## 5 Metatheory

System F is type sound, and the proof follows the same structure as for the simply-typed $\lambda$-calculus.

**Theorem** (Type soundness)**.** *If $\emptyset, \cdot \vdash e : \tau$ and $e \to^* e'$ and $e' \not\to$, then $e'$ is a value.*

As before, we prove type soundness using progress and preservation. The canonical forms lemma gains a case for polymorphic types: if $\emptyset, \cdot \vdash v : \forall X.\, \tau$, then $v$ is a type abstraction $\Lambda X.\, e$. The preservation proof needs a new substitution lemma, for substituting types for type variables, which is used in the case for Type-$\beta$:

**Lemma** (Type substitution)**.** *If $\Delta \cup \{X\}, \Gamma \vdash e : \tau$ and $\Delta \vdash \tau'$ ok, then $\Delta, \Gamma\{\tau'/X\} \vdash e\{\tau'/X\} : \tau\{\tau'/X\}$.*

Like the simply-typed $\lambda$-calculus, System F is strongly normalizing.

**Theorem** (Strong normalization, Girard 1972)**.** *If $\emptyset, \cdot \vdash e : \tau$, then every sequence of reductions starting from $e$ is finite.*

The proof is much harder than for the simply-typed $\lambda$-calculus, because the type of $\Lambda X.\, e$ quantifies over *all* types, including the type $\forall X.\, \tau$ itself. Girard's proof uses a technique called *reducibility candidates*. A consequence is that System F is still not Turing complete, and $\Omega$, Y, and Z are still not typable. But System F is very expressive: Girard also showed that the functions on natural numbers that can be written in System F are exactly those that can be proved total in second-order arithmetic, which includes far more than the primitive recursive functions.

## 6 Type Erasure

The semantics presented above passes types around explicitly, but types never affect the result of a program. In an implementation, we can eliminate the types before running the program. The following translation *erases* the types from a System F expression:

$$\begin{aligned} \mathit{erase}(x) &= x \\ \mathit{erase}(\lambda x : \tau.\, e) &= \lambda x.\, \mathit{erase}(e) \\ \mathit{erase}(e_1\ e_2) &= \mathit{erase}(e_1)\ \mathit{erase}(e_2) \\ \mathit{erase}(\Lambda X.\, e) &= \lambda z.\, \mathit{erase}(e) \qquad \text{where } z \text{ is fresh for } e \\ \mathit{erase}(e\ [\tau]) &= \mathit{erase}(e)\ () \end{aligned}$$

The other cases simply erase the types in their subexpressions. A type abstraction becomes a function of a dummy argument $z$, and a type application passes it the dummy argument $()$. So type abstractions are still values, and each TYPE-$\beta$ step becomes an ordinary $\beta$ step:

$$\mathit{erase}((\Lambda X.\, e)\ [\tau]) = (\lambda z.\, \mathit{erase}(e))\ () \longrightarrow \mathit{erase}(e) = \mathit{erase}(e\{\tau/X\}).$$

The following theorem says that the translation is adequate.

**Theorem** (Erasure)**.** *For all expressions $e$ and $e'$, we have $e \longrightarrow e'$ if and only if $\mathit{erase}(e) \longrightarrow \mathit{erase}(e')$.*

So we can type check a program with all of its types, and then run it without them, as OCaml does.

## 7 Parametricity

What can a function of type $\forall X.\ X \to X$ do? It knows nothing about the type $X$, so it cannot inspect a value of type $X$, create a new one, or modify one. All it can do is return its argument, so it must be the identity function! Similarly, a function of type $\forall X.\ X \to X \to X$ can only return one of its two arguments, so there are exactly two such functions (up to equivalence): the Church Booleans. And there is no closed term of type $\forall X.\ \forall Y.\ X \to Y$ at all, since there is no way to produce a value of an arbitrary type $Y$.

This phenomenon is called *parametricity*. Reynolds (1983) showed that every polymorphic function treats its type arguments uniformly, and Wadler (1989) showed how to use this to derive "theorems for free" from types alone. For example, every function $f$ of type $\forall X.\ X$ **list** $\to X$ **list** (in a language with lists) can only rearrange, drop, or duplicate the elements of its argument, so it commutes with mapping any function $g$ over a list: $\mathsf{map}\ g\ (f\ l) = f\ (\mathsf{map}\ g\ l)$.

## 8 Type Inference

System F requires programmers to write type abstractions, type applications, and type annotations explicitly, which quickly becomes tedious. Could we infer them instead? The *type reconstruction* problem asks whether, for a given untyped $\lambda$-calculus expression $e'$, there exists a well-typed System F expression $e$ such that $\mathit{erase}(e) = e'$. Wells showed in 1994 that this problem is undecidable. In the next lecture, we will see a restricted form of polymorphism, *let-polymorphism*, for which type inference is decidable, and which is the basis of the type systems of OCaml and other languages in the ML family.